8 Contract testing for event-driven architecture

This chapter covers

- Introduction to asynchronous API testing
- Pact consumer message setup with Kafka
- Kafka consumer implementation
- Event handler consumer test implementation
- Event consumer contract generation
- Kafka producer implementation
- Event producer test implementation

Chapter 4 covered the implementation of consumer-driven contract testing (CDCT) with APIs. However, if you work with an event-driven architecture, you'll want to understand how to implement CDCT with messages. In this chapter, we'll use Node.js and Kafka to walk through the implementation steps of CDCT in an event-driven application, from setting up the project and applying best practices to generating the consumer contract.

If you understand the foundational concepts from chapter 3, it's time to start implementing contract testing. In our experience of working with event-driven applications within a microservices environment, it's important to take care of event consumers interacting with the service. Event messages are often provided to update many consumers asynchronously. Ensuring no breaking changes occur within the message contracts is key to providing a seamless experience for the services consuming the events.

To provide more context regarding why contract testing is relevant in the world of events, consider the following: The structure of the event messages can evolve throughout the development process; therefore, the